

The Rover Fleet Project: A Guided Tour

Constrained graph pruning + trust-aware GNN task assignment,
explained simply — and then in full detail, with the real numbers

Companion document to the paper *Constrained Edge Pruning and Trust-Aware GNN Inference for Task*

September 2026 • all numbers from the repository’s own experiment logs

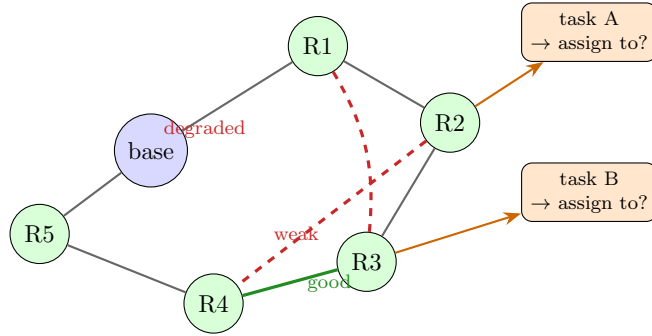
How to read this document. Every number in it is real: it comes from the experiment logs committed next to the code (`benchmark.jsonl`, `benchmark_bf0.75.jsonl`, `sweep*.jsonl`, `worked_example.json`) or from the paper’s regression harness. Nothing is invented, rounded in our favor, or cherry picked — the embarrassing numbers are in here too, because they are the interesting ones. Each section has a *simple* version (what to say in the meeting) and a *detail* version (what to say when someone digs in).

Contents

1	The problem in one picture	2
2	The pipeline, start to finish	3
3	The core idea, on a graph small enough to check by hand	3
3.1	The instance	3
3.2	Three ways to be wrong, and one way to be right	3
3.3	The hand-checkable arithmetic	4
3.4	Three independent solvers, one answer	5
4	The hidden pattern: why generic QUBO solvers fail here	5
5	How the benchmark works (and how it audited itself)	6
5.1	The measurement design	6
5.2	The metrics, defined	7
5.3	The audit caught the audit	7
6	Results I: the honest scoreboard	7
6.1	Reading the scoreboard like a reviewer	8
7	Results II: the constraint-pressure sweep — does the guarantee ever pay?	9
8	Results III: the assignment layer — where the project genuinely wins	11
9	Results IV: scale — where the story has to end up	12
10	The full scorecard	12
11	Reproducibility — how to show any of this live	12
12	The one-slide cheat sheet	13

1 The problem in one picture

Simple version. A team of rovers explores an area with no infrastructure. They talk to each other over short-range radio links that appear and disappear as they move. Every cycle, three things must happen: **(1)** decide which radio links are worth using, **(2)** decide which rover does which task, and **(3)** actually deliver the results back to base — over the links you kept. Bad link choices silently break (3); bad task choices waste the fleet. This project builds a system that makes both decisions automatically, under hard rules (“no rover talks to more than D others”, “don’t spend more compute than the budget”), and — unusually — *measures itself honestly* to find out whether the fancy optimization is actually worth it.



Every cycle the graph is different. Results must flow back to base over the *widest* path (weakest link on the path sets the success probability), so one bad link on the only path home can silently kill a task.

Figure 1: The world: 10 rovers (here 5 for legibility), links that come and go, tasks that must be assigned, and a base that must receive the results.

Detail version. The simulator (10 rovers, 20-cycle episodes, 3 tasks per cycle, drifting per-rover reliability) generates a dynamic candidate graph each cycle. Each candidate link carries six normalized features measured online: *trust* (an exponentially-weighted estimate of that rover’s past delivery behavior), *link quality*, *battery*, *proximity*, *freshness* (how recent the observation is), and *task criticality*, plus a *processing cost*. Two decisions are made per cycle:

1. **Pruning** — choose a subset of links to keep, subject to hard constraints: per-node degree caps, a global processing budget, and exact connectivity of the mission-critical node set to the base. This is formulated as a QUBO (quadratic unconstrained binary optimization) and solved in the cloud tier.
2. **Assignment** — give each task to the rover with the best expected outcome, scored by a trust-aware graph neural network running on the pruned graph, in the fog tier.

Mission success is *causally* tied to pruning: a task’s result is delivered with probability equal to the widest-path bottleneck (the weakest link on the best path) from the assigned rover to base *on the kept graph*. Prune badly and delivery fails — the metric cannot be gamed.

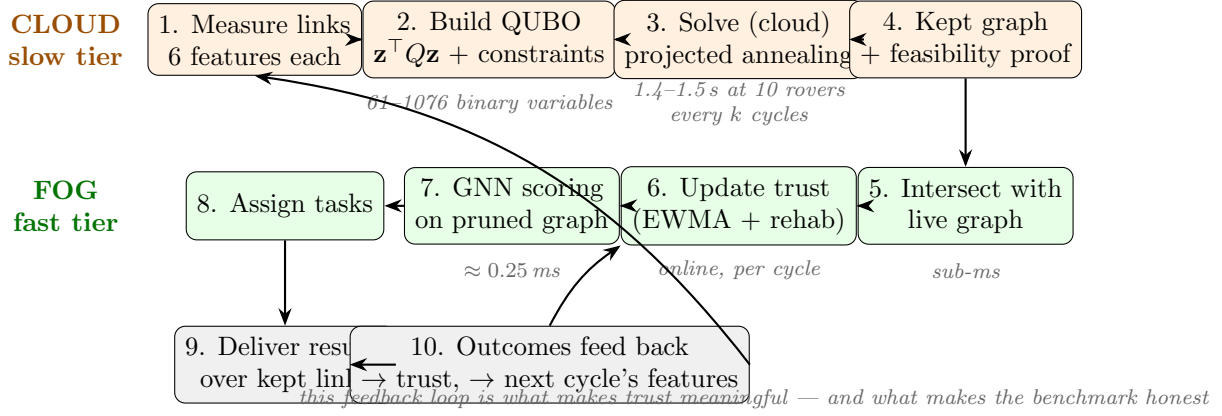


Figure 2: The full loop. The expensive optimization runs rarely, in the cloud; the cheap decisions run every cycle, on-device. Outcomes feed back into trust, so both layers learn from the same reality.

2 The pipeline, start to finish

The one-line version for slides. *Measure the network → encode “which links to keep” as a constrained QUBO → solve it in the cloud → run cheap trust-aware GNN assignment on the surviving graph → let outcomes retrain trust.*

Real measured costs per stage (10 rovers, medians over 16 seeds × 20 cycles):

stage	measured time	note
pruning decision (threshold)	0.0065 ms	what we’re compared against
pruning decision (top- K)	0.0075 ms	
pruning + repair (greedy)	0.14–0.33 ms	constraint-aware baselines
QUBO solve (projected annealing)	1470 ms	cloud tier, amortized
GNN inference (pruned graph)	≈ 0.25 ms	fog tier, every cycle

Table 1: Per-decision wall clock. The QUBO solve is ≈ 5,800× the GNN inference it protects — which is exactly why it belongs in the cloud tier and why the paper refuses to claim a latency win.

3 The core idea, on a graph small enough to check by hand

This section is the heart of the story. Everything the big system does is already visible in a 5-rover, 7-link example that we can solve *exactly, by hand*, *64 subsets at a time*. Every number below was computed by the actual code (`worked.example.py`) and cross-checked three independent ways.

3.1 The instance

Five rovers: base 0, and rovers {1, 2, 3, 4}. Seven candidate links. The mission requires rovers {2, 3, 4} to stay connected to the base (rover 1 is a relay, not required). Rules: no rover may use more links than its degree cap; the kept links’ total processing cost must fit in a budget of 12 units; hard link 0–1 is always kept (it is rover 1’s only tether).

3.2 Three ways to be wrong, and one way to be right

There are $2^6 = 64$ possible keep/drop choices for the six optional links. The code evaluated *all 64*, checking constraints and computing the exact QUBO energy of each. Result:

link	trust	link q.	battery	prox.	fresh.	task	utility	cost
0-1 (hard)	.90	.90	.80	.70	.90	.50	0.810	2
0-2	.90	.90	.90	.90	.90	.90	0.900	1
0-3	.40	.50	.50	.40	.40	.20	0.405	8
1-2	.80	.70	.90	.60	.80	.40	0.705	3
2-3	.70	.80	.80	.70	.90	.60	0.750	4
2-4	.85	.85	.75	.80	.85	.55	0.790	2
3-4	.60	.60	.70	.50	.60	.30	0.555	6

Table 2: The instance. Utility = weighted sum of the six features (weights 0.25/0.25/0.10/0.10/0.15/0.15 — they sum to 1). Cost is quantized to integer budget units ($\times 10$). The previous cycle kept $\{0-1, 0-2, 1-2, 2-3, 3-4\}$.

outcome	count	meaning
fully feasible subsets	3	only 4.7% of choices are legal
connectivity traps	30	look great, strand a required rover
degree/budget violators	31	break the hard rules

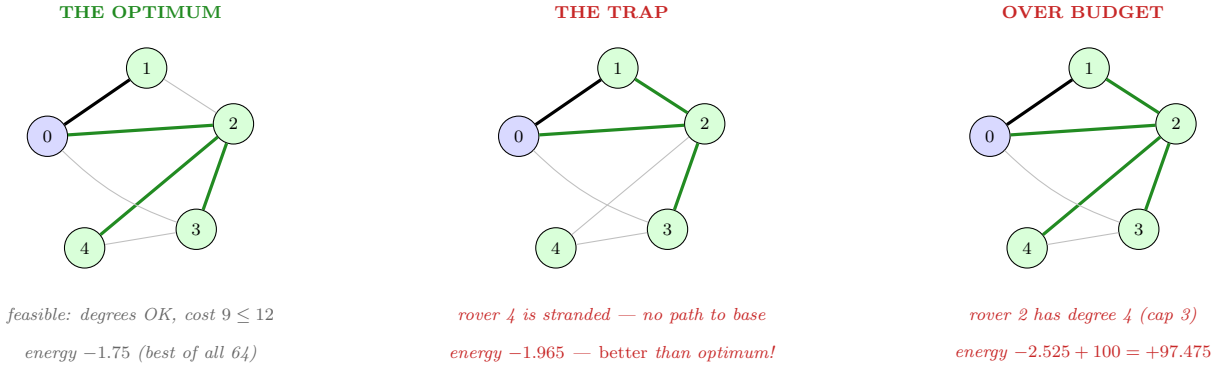


Figure 3: The three most tempting subsets in the example. Left: the true optimum. Middle: the trap — a naive “keep the best-looking links” pruner takes it, because its energy (-1.965) is *lower* than the optimum’s (-1.75), and nothing in its field of view says rover 4 is stranded. Right: the constraint-breaking variant, which the QUBO’s penalty terms price at $+97.475$.

3.3 The hand-checkable arithmetic

Optimum = $\{0-1, 0-2, 2-3, 2-4\}$:

- utility of kept optional links: $0.900 + 0.750 + 0.790 = -2.440$
- sparsity charge: $3 \times 0.08 = +0.240$
- switching charge vs. last cycle: 2-3 stays (0), 2-4 switches on ($+0.15$), 1-2 switches off ($+0.15$), 3-4 switches off ($+0.15$) = $+0.450$
- **energy** = $-2.440 + 0.240 + 0.450 = \mathbf{-1.750}$

(The hard link 0-1 contributes $-0.810 + 0.080 = -0.730$ to the QUBO’s *constant* term — identical for every subset, so it never changes the ranking.) Feasibility: cost $2 + 1 + 4 + 2 = 9 \leq 12$; degrees 3, 2, 3, 2, 2 all within caps; rovers $\{2, 3, 4\}$ all reach the base. ✓

The trap = $\{0-1, 0-2, 1-2, 2-3\}$: utility $0.900 + 0.705 + 0.750 = 2.355$; sparsity $3 \times 0.08 = 0.240$; switching: only 3-4 turns off ($+0.15$) $\Rightarrow -2.355 + 0.240 + 0.150 = \mathbf{-1.965}$ — *better* than the optimum, and degrees (2, 2, 3, 2) and cost ($10 \leq 12$) are perfectly legal. Its only flaw is invisible to a utility-only pruner: **rover 4 has no path to the base**. That single number is the entire motivation for constraint-aware pruning.

3.4 Three independent solvers, one answer

method	what it does	result
exhaustive enumeration	all 64 subsets, closed-form slacks	-1.750
projected annealer (production)	anneals over <i>feasible</i> subsets, cold start	-1.750 exact
MILP / CBC (exact baseline)	linear program with integer links	-1.750

And the Q-matrix cross-check: the energy of the optimum’s binary vector computed straight from the exported 61×61 matrix Q equals the enumeration energy to 10^{-9} . *The encoding is not just plausible; it is verified.*

Where do 61 variables come from, on a 5-rover graph with 6 decisions? 6 edge variables x_e + 13 degree-slack bits + 28 flow bits + 14 flow-capacity bits = 61. The extra 55 are the price of encoding *hard* constraints (degree, budget, and connectivity-via-single-commodity-flow) inside a purely binary quadratic form. This is also the seed of the paper’s central discovery — see Section 4.

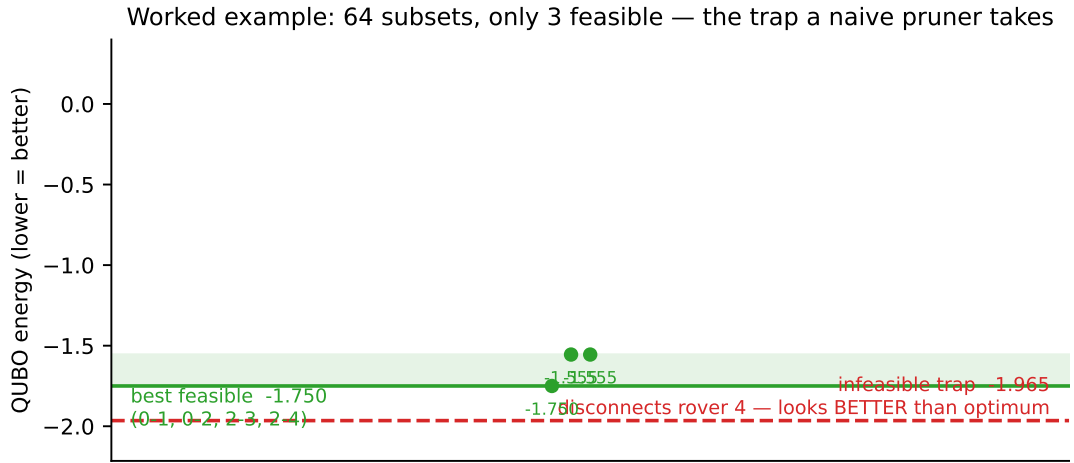


Figure 4: The energy landscape of the worked example. Only three subsets are feasible (green band); the most tempting infeasible one (red) *beats* the optimum on raw energy. A pruning method without constraint awareness doesn’t just risk being illegal — it actively prefers the wrong answer.

4 The hidden pattern: why generic QUBO solvers fail here

The discovery. On this penalty-encoded landscape, serious annealing solvers fail not because they are weak but because of the *shape* of the energy surface. This is the paper’s most cited-worthy finding, and it has a clean one-sentence explanation.

The one-sentence explanation. From any feasible solution, every single-bit flip either (a) breaks a constraint and costs at least 100 energy points, or (b) stays feasible and changes the score by at most about 1.2. The barrier is $\geq 80\times$ taller than the largest beneficial move — so at any temperature where good moves are accepted, bad moves are never accepted, and the sampler *can never leave a feasible basin*. It is trapped by design.

Empirically, on the actual Q matrices of 25 exactly-verifiable instances:

solver	exact-solve rate
simulated quantum annealing (OpenJij)	48%
local bit-flip simulated annealing	56%
D-Wave neal	60%
tabu search	64%
exact linearized solve (CBC)	100%
projected annealer (ours, engineering note)	100%

Supporting evidence: random restarts of bit-level annealing returned *infeasible* states in 19 of 25 instances. The solvers aren’t converging to wrong optima — they are failing to even find legal ones.

The fix (and its honest caveat). The production solver, *projected edge annealing*, stops sampling bit strings and instead toggles whole edges, projecting every candidate back to full feasibility (exact slack recomputation + spanning-tree flow) before scoring it. It solves 100% of the verifiable instances ($\approx 2.4 \times 10^4$ energy evaluations, 2.1 s per solve) and stays within 10^{-3} of the MILP optimum on 20 larger instances (95% exact). The caveat, which the paper states plainly: its move set is richer than the bit-samplers’, so it is an *engineering* result, not a scientific one — and on classical hardware, CBC solves the same instances exactly in ≈ 15 ms, about $100\times$ faster. *The QUBO’s remaining job is to be the input format for quantum hardware, not to win on a CPU.*

5 How the benchmark works (and how it audited itself)

5.1 The measurement design

Simple version. We run the *identical* simulated world through six pruning strategies — same rovers, same drift, same tasks, seed by seed — and compare what happens to success, feasibility, and cost. Because every arm sees the same world, differences are causal, not luck.

Detail. 6 arms $\times$ 16 seeds $\times$ 20 cycles per configuration, two budget regimes (nominal fixed $K=15$; binding $K=0.75\times$ the full-graph cost), plus three constraint-pressure sweeps (Section 7). In total: **1,152 scenario cells, $\approx 23,000$ simulated rover-cycles**. Paired statistics (per-seed differences, bootstrap 95% CIs, sign-flip permutation tests) because across-seed comparisons understate paired differences.

The six arms:

1. **full graph** — prune nothing (the do-nothing baseline);
2. **threshold** — keep every link whose utility beats a cutoff (constraint-blind);
3. **top- K** — keep the K best links by utility (constraint-blind);
4. **threshold+repair** — same greedy prune, then the *same feasibility repair the QUBO gets*: enforce the same degree caps, the same budget, the same mission connectivity;
5. **top- K +repair** — same with top- K ;
6. **QUBO (ours)** — the constrained optimization.

Arms 4–5 exist because of a review insight worth repeating: *a violations table against constraint-blind opponents is a straw man* — threshold and top- K have no budget or degree parameters, so of course they “violate” things. The repaired variants are given every constraint the QUBO has, plus the same repair pass. Any remaining QUBO advantage in feasibility is then real, not manufactured.

5.2 The metrics, defined

metric	definition (as implemented)
task success	delivered results / assigned tasks, per cycle, averaged over the episode
offload rate	task slots actually assigned / total task slots — a pruner that disconnects rovers <i>strands tasks</i> and this catches it
delivery-failure rate	failed delivery attempts / attempted deliveries — catches pruners that keep <i>weak paths</i>
pruning precision	fraction of kept edges that are delivery-critical
pruning recall	fraction of delivery-critical edges that were kept
delivery-critical (ground truth)	an edge is critical iff removing it from the <i>full</i> candidate graph strictly reduces the widest-path delivery bottleneck of any mission-critical node — recomputed per cycle, a true counterfactual
violations	mean counts per 20-cycle run: per-node degree excess, budget units overrun, mission nodes unreachable
relaxation bookkeeping	every solve logs which constraint level it ran at (full / no-budget / budget-only / unconstrained / mission-unreachable) — the audit trail that caught finding 5.3 below

5.3 The audit caught the audit

At the paper’s *nominal* fixed budget $K=15$, the budget is below the cost of *any* connected subgraph at 10 rovers — so it can never be satisfied. The solver’s relaxation ladder responded by silently running **319 of 320 cycles without the budget constraint**. The original comparison table’s QUBO row was, unknown to us, effectively unconstrained. The bookkeeping we built to prove feasibility is what exposed this.

Two consequences, both now in the paper:

1. All constraint-relevant comparisons were re-run at a *binding, feasible* budget ($K = 0.75 \times$ full-graph cost), where the QUBO solves in full-constraint mode in **320 of 320 cycles** with **zero violations**.
2. The fairness correction cuts both ways: against the constraint-aware repaired baselines, the QUBO’s success deficit shrinks from -4.8pp (vs. blind threshold) to -4.1pp (vs. threshold+repair) — and at the nominal (infeasible) budget it was only -2.1pp and not significant ($p=0.10$). Honest comparisons need honest opponents.

6 Results I: the honest scoreboard

arm (binding budget)	success	offload	del.-fail	solve (ms)	kept	prec. / rec.
full graph	36.0%	100%	38.5%	—	100%	.089 / 1.000
threshold	37.0%	95.5%	34.0%	< 0.01	84%	.097 / .931
top- K	36.4%	89.8%	30.9%	< 0.01	50%	.109 / .562
threshold+repair	35.3%	99.6%	38.3%	0.33	44%	.177 / .817
top- K +repair	35.4%	99.7%	39.3%	0.14	42%	.187 / .824
QUBO (ours)	31.3%	99.8%	42.7%	1470	53%	.116 / .673
violations per run (degree / budget / connectivity):						
full	158.8 / 20.0 / 0.4	threshold	131.7 / 12.8 / 2.3	top- K :	53.6 / 0 / 10.0	
thr+rep	3.1 / 0.3 / 0.8	top+rep	1.1 / 0 / 0.9	QUBO:	0 / 0 / 0.4	

Table 3: The full scoreboard at the binding budget. Connectivity misses of ≈ 0.4 per run appear in *every* arm including the full graph: those are cycles where the live network itself cannot reach a required node — nobody can fix those. Trust metrics: Spearman 0.295 (QUBO, best) vs 0.251–0.271; AUC 0.681 vs 0.624–0.683.

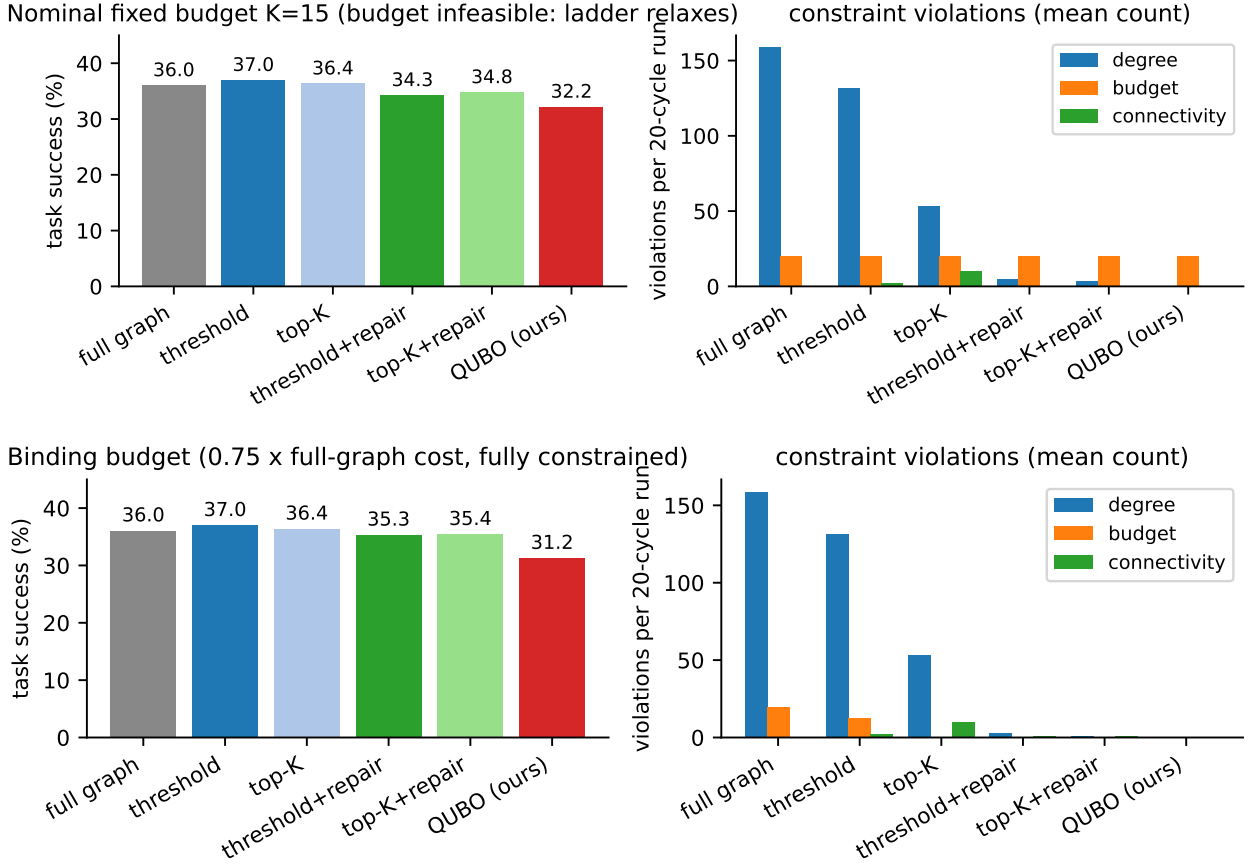


Figure 5: Top: the nominal-budget benchmark (budget infeasible — every arm’s budget “violations” here reflect the infeasible $K=15$, and the QUBO’s ladder relaxes it silently). Bottom: the binding-budget benchmark, where the QUBO is fully constrained and the guarantees are real. Success (left) and violations (right), 16 seeds $\times$ 20 cycles.

6.1 Reading the scoreboard like a reviewer

Pattern 1 — two different failure modes. Top- K has the *best* delivery-failure rate (30.9%) and the *worst* offload (89.8%): it prunes so aggressively that it disconnects rovers and *strands tasks* — the deliveries it does attempt are over strong links, but 10.2% of tasks never even launch. The QUBO is the mirror image: near-perfect offload (99.8%, nobody stranded) but the *worst* delivery-failure (42.7%) — it keeps everyone connected over paths that include weak links, because its soft objective misranks delivery-critical edges (critical recall 0.673 vs 0.817–0.824 for the repaired baselines). Same metric table, two completely different diseases.

Pattern 2 — the guarantee is real and it is the only thing that is. At the binding budget the QUBO is the *only* arm with zero degree and budget violations. The repaired baselines come close (1–3 degree violations per run) but not exact; the blind baselines don’t even try (threshold: 131.7 degree violations, 12.8 budget overruns per run).

Pattern 3 — the cost of the guarantee, priced. Paired per-seed differences at the binding budget: QUBO – threshold+repair = -4.1pp , CI $[-6.4, -1.9]$, $p = 0.005$; QUBO – top- K +repair = -4.2pp , CI $[-7.0, -1.5]$, $p = 0.016$; QUBO – top- K = -5.1pp , $p = 0.006$. The guarantee costs about 4 success points at this scale, and the measurement is tight enough to know it’s not noise.

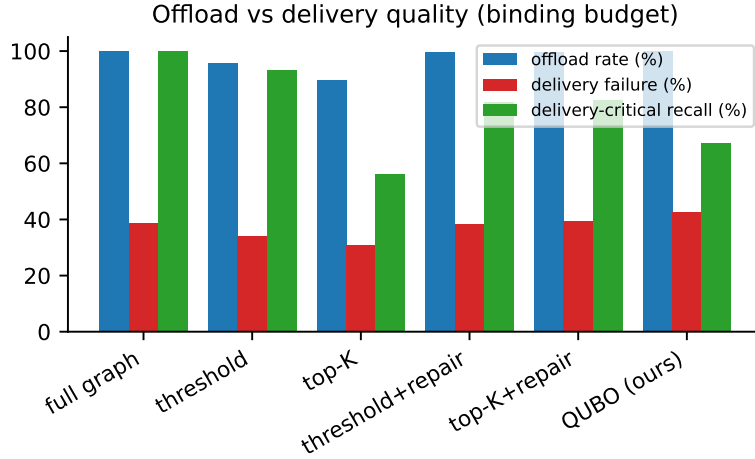


Figure 6: The three delivery metrics side by side (binding budget): the top- K arms trade stranded tasks (offload 89.8%) for strong paths (delivery-fail 30.9%); the QUBO is the mirror image. The repaired baselines sit in between on both.

Whether that price is worth paying is exactly what the next section measures.

Pattern 4 — trust quality, with the confound flagged. The QUBO arm posts the best Spearman correlation with hidden reliability (0.295 vs 0.251–0.271). But the AUC story is a tie at best (0.681 vs 0.683 for full/top- K), and — more importantly — these numbers are measured on outcomes observed *under each pruner’s own policy*. Different pruners create different observation streams, so the cross-arm trust comparison is selection-confounded. We say so in the paper rather than harvest the flattering number.

7 Results II: the constraint-pressure sweep — does the guarantee ever pay?

The question the whole paper hinges on. “The QUBO gives guarantees” is only a selling point if there is a regime where *violating* the constraints actually hurts. So we made the constraints progressively harder and watched for the crossover.

Design: one constraint tightened at a time, ten levels, 16 seeds per cell, all six arms — budget as a fraction β of the full-graph cost ($\beta \in \{0.35, 0.5, 0.75, 1.0\}$), degree cap $D \in \{2, 3, 4\}$, mission-critical roster $|R| \in \{3, 5, 8\}$.

constraint	level	QUBO	best repaired	QUBO – best (paired)
budget β	0.35	34.6%	34.7%	–0.1pp, $p = 1.00$ (tie)
budget β	0.50	31.6%	35.2%	–3.7pp, $p = 0.033$
budget β	0.75	31.3%	35.3%	–4.1pp, $p = 0.004$
budget β	1.00	31.7%	35.3%	–3.7pp, $p = 0.020$
degree D	2	30.9%	34.3%	–3.3pp, $p = 0.15$
degree D	3	32.2%	34.3%	–2.1pp, $p = 0.10$
degree D	4	33.3%	34.8%	–0.9pp, $p = 0.52$
roster $ R $	3	32.2%	34.3%	–2.1pp, $p = 0.10$
roster $ R $	5	32.4%	35.1%	–2.7pp, $p = 0.058$
roster $ R $	8	34.6%	37.8%	–3.2pp, $p = 0.020$

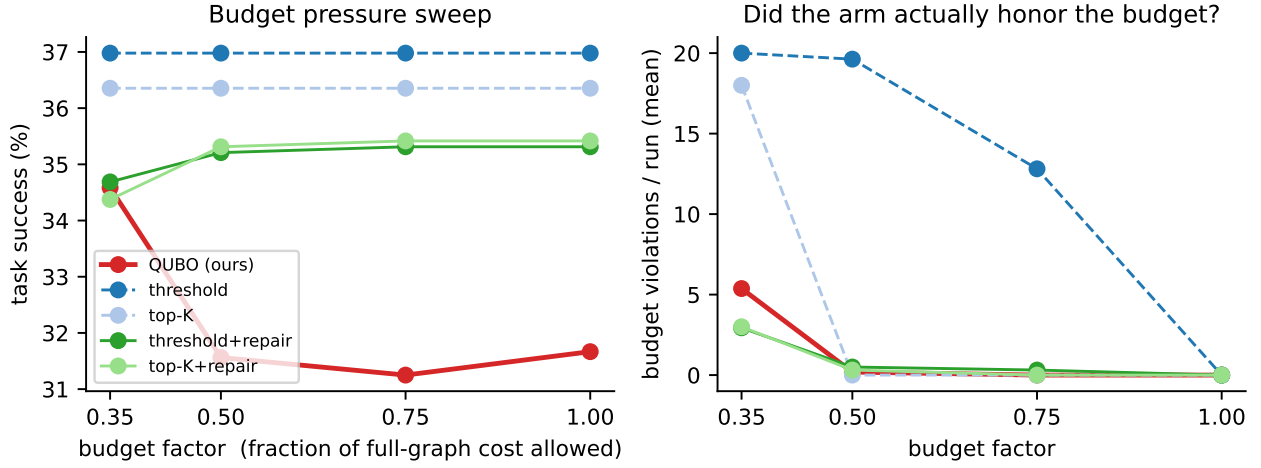


Figure 7: Budget pressure. Left: success. Right: budget violations per run — the blind threshold pruner violates the budget in 20 of 20 cycles at $\beta=0.35$ and still 12.8 of 20 at $\beta=0.75$, while the QUBO converges to zero (its small residual at $\beta \leq 0.5$ is the ladder relaxing a genuinely infeasible request, which the bookkeeping counts and reports).

(In the degree and roster sweeps the budget stays at the nominal fixed $K=15$, which is infeasible — that is why every arm shows ≈ 20 budget violations there; the swept constraint is the one on the x-axis.)

The answer, and it is not the one the pitch predicts. At extreme budget pressure ($\beta=0.35$) the QUBO *ties* the best repaired baseline (-0.1pp , $p=1.0$) — there, even the baselines’ repair passes leave residual violations (2.9–3.0 per run) and the gap closes to noise. Everywhere else the QUBO loses 2–4 points, and the gap *narrows* as constraints loosen. **There is no crossover.** In this simulator, honoring constraints exactly never converts into a success advantage — even when constraint-blind pruners violate the budget in literally every cycle. The honest conclusion: *the guarantees are insurance. They are worth their premium exactly when violations are disqualifying (safety, contractual, regulatory), not when success is the only currency.* That is a defensible product claim, and the sweep is what makes it defensible.

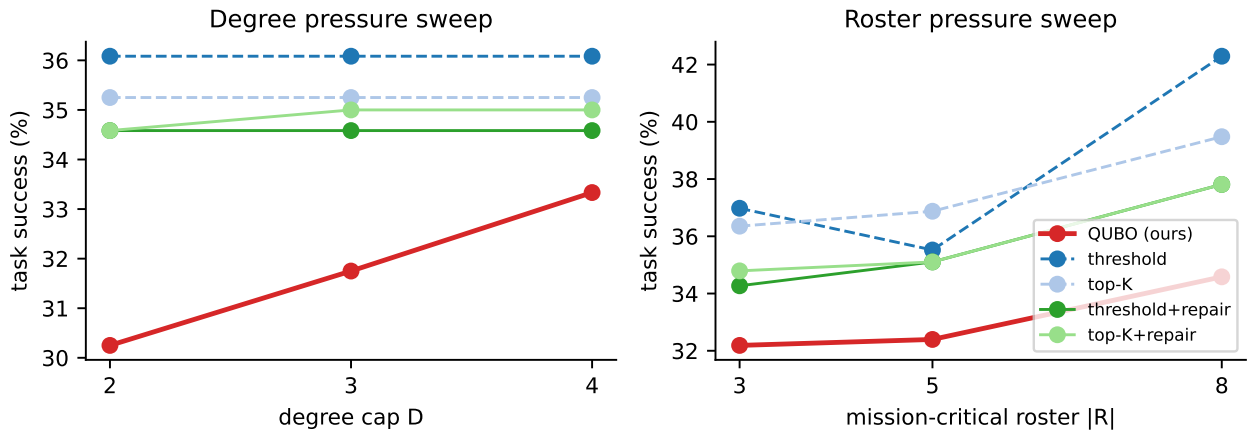


Figure 8: Degree and roster pressure: same story — the QUBO’s deficit narrows as the constraint loosens, never reverses.

8 Results III: the assignment layer — where the project genuinely wins

This is the cleanest positive result in the project, and it is about the *GNN + trust* layer, not the optimizer.

Four assignment policies, identical worlds, 8 seeds:

policy	success
random among eligible rovers	26.0%
trust-only (bandit-style baseline)	31.7%
no-graph logistic readout	38.1%
trust-aware GNN (ours)	39.8%
GNN – trust-only	+8.1pp, CI [+4.2, +11.7], $p = 0.016$ significant
GNN – no-graph	+1.7pp, CI [−2.1, +6.9], $p = 0.736$ not significant

How to present this without overclaiming. The GNN beats the strong simple baseline by 8.1 points with a clean paired statistic — that’s real and headline-worthy. But the margin over a no-graph readout (with the same features, no message passing) is +1.7pp and statistically indistinguishable from zero. So the honest attribution is: *most of the value is the trust model and the path-aware gating; message passing adds a small, unproven extra at ten-rover density*. The paper says exactly this. A reviewer is far more likely to believe the +8.1 when you volunteer the +1.7.

Two more results in this layer:

- **Training the readout on the unbiased exploration subset (78 samples) did not help:** 36.7% trained vs 32.8% fixed weights, difference −3.9pp, CI [−11.7, +11.7], $p = 0.49$. Reported as the null result it is.
- **Fault recovery works:** under a scripted fault (most reliable rover degraded + packet-loss spike), success returns to its pre-fault baseline within 3–4 cycles after the fault expires, with the edge lifecycle (active → suspect → probation → removed) keeping pruned links observable.

And the cloud-amortization question — can we solve rarely instead of every cycle?

solve interval k	success	solves/cycle	amortized solve
1	30.3%	1.00	1279 ms
2	29.4%	0.50	618 ms
4	26.9%	0.25	273 ms
8	31.9%	0.15	151 ms

Reusing a two-cycle-old decision costs under one success point while halving the solve load; beyond that the measurement is dominated by seed variance (6 seeds), which the paper states rather than drawing a curve through noise.

9 Results IV: scale — where the story has to end up

n	$ E $	QUBO vars	solve (s)	E projected	E greedy	E MILP
$10^\dagger$	30	427	12.6	−6.39	—	−6.39
$20^\dagger$	63	1076	41.4	−15.05	−11.76	−15.36
30	95	194	15.4	−21.12	−16.62	—
50	169	328	94.5	−36.71	−24.30	—

(† connectivity enforced only for $n \leq 20$ — stated, not hidden. At $n=20$ the projected annealer is within 2.0% of the exact MILP optimum; against the repaired greedy it finds 27% better energy at $n=30$ and 51% better at $n=50$.)

The scale story in three sentences. The dense Q matrix grows quadratically (1,076 variables at just 20 rovers), so the formulation as built tops out at tens of rovers — a stated limitation, not a surprise. Solve time grows into the tens of seconds, which is fine only because the architecture amortizes it in the cloud. And the constrained objective genuinely shines against greedy at scale (+51% better energy at $n=50$) — though at those sizes the wins belong to “the constrained objective + feasible-space search,” not to QUBO *solving* per se, since CBC is both exact and $\approx 100\times$ faster wherever it can reach.

10 The full scorecard

Where the project wins

Formulation correctness: encoding verified against independent re-derivation, 64-subset enumeration, and CBC — three methods, one answer, 10^{-9} agreement.

The penalty-barrier discovery: a quantified, reproduced explanation ($\geq 80\times$ barrier ratio) of why annealing samplers fail on penalty-encoded QUBOs (48–64% exact).

Feasibility guarantees: the only arm with zero violations at every tested pressure level; blind baselines violate the budget in up to 20/20 cycles.

Assignment layer: +8.1pp over trust-only ($p=0.016$), +13.8pp over random, with paired statistics.

Offload guarantee: 99.8% of tasks actually assigned (top- K : 89.8% — it strands tasks).

Audit culture: the harness caught its own infeasible budget (319/320 relaxed cycles) and forced an honest re-run.

Where it loses

Raw success at 10 rovers: −4.1pp vs repaired baselines ($p=0.005$), −5.1pp vs blind top- K .

Classical speed: CBC is exact and $\approx 100\times$ faster than the annealer; the QUBO detour buys nothing on a CPU.

No success crossover: even at extreme pressure the QUBO only *ties* (−0.1pp, $p=1.0$); the guarantee never converts into success in this simulator.

Message passing unproven: +1.7pp over a no-graph readout, $p=0.736$.

Delivery quality: worst delivery-failure rate (42.7%) and lowest critical recall (0.67) — the soft objective misranks delivery-critical edges.

Small-scale economics: a 1.47s solve protecting 0.25ms of inference ($\approx 5,800\times$) only makes sense amortized in the cloud, or at scale.

11 Reproducibility — how to show any of this live

to demonstrate	run
the worked example (all 64 subsets, 3-way agreement)	<code>python worked_example.py</code>
the pruner benchmark (both budgets, resumable)	<code>python benchmark_full.py [start] [end] [budget]</code>
the constraint-pressure sweeps (resumable)	<code>python constraint_pressure.py budget degree </code>
extract every quoted number	<code>python analyze_results.py</code>
regenerate every figure in this document	<code>python make_figures.py</code>
the 486-check regression suite	<code>python battle_test.py</code>

Every number in this document traces to a JSONL artifact or the worked example; `make_figures.py` writes `doc_data.json` containing each quoted aggregate so text and figures cannot drift apart.

12 The one-slide cheat sheet

Ten numbers to memorize for the talk:

1. 64 subsets → only **3 feasible**; the best-looking illegal one scores -1.965 vs the true optimum -1.75 — constraints change the answer, not just check it.
2. The QUBO for that toy has **61 variables** (6 edges + 55 constraint bookkeeping) — constraints are expensive to encode, which is the seed of the barrier problem.
3. Solver audit: annealers solve **48–64%** of verifiable instances; barrier ratio $\geq 80\times$; projected annealer and CBC: **100%**.
4. CBC: exact in ≈ 15 **ms**; annealer 2.1 s — the QUBO’s job is the hardware interface, not classical speed.
5. Production solve: **1.47 s** at 10 rovers vs threshold’s **0.007 ms**; GNN inference **0.25 ms** ($\approx 5,800\times$ ratio) $\Rightarrow$ cloud amortization is mandatory ($k=8$: 151 ms, 31.9%).
6. Success: QUBO **31.3%** vs threshold **37.0%** at 10 rovers (-4.1pp vs repaired baselines, $p=0.005$) — the honest price of guarantees at small scale.
7. The guarantee itself: **zero** degree/budget violations at the binding budget; blind threshold violates the budget **20/20 cycles** at extreme pressure.
8. Constraint-pressure sweep: **no crossover** — tie at extreme pressure (-0.1pp , $p=1.0$), 2–4pp deficit elsewhere. Guarantees are insurance.
9. Offload vs delivery: QUBO offloads **99.8%** (top- K 89.8%) but has the worst delivery-failure (**42.7%** vs 30.9%) — two different failure modes, both measurable.
10. Assignment layer: **+8.1pp** over trust-only ($p=0.016$) — the project’s cleanest win; message passing itself: **+1.7pp** ($p=0.74$), honestly reported as unproven.

slide	claim	evidence to show
1	the problem	the dynamic-graph picture (Fig. 1)
2	the pipeline	Fig. 2 with timings
3	pruning is constrained optimization	worked example + trap figure (Fig. 3)
4	the encoding is verified	3-way agreement, 10^{-9}
5	generic QUBO solvers fail	audit bar chart, barrier explanation
6	our solver + honest CBC comparison	100% exact, 2.1 s vs 15 ms
7	the benchmark design	6 arms, paired stats, metric definitions
8	the honest scoreboard	Fig. 5, Table 3
9	does the guarantee ever pay?	budget-sweep figure, “no crossover”
10	what actually wins	+8.1pp assignment, offload guarantee, audit